

Projeto Final - Tema Livre

Projeto (*valor 0 Pontos*)

Defesa Individual (*valor 5 Pontos*)

O projeto consiste no desenvolvimento de um sistema em equipes de até **5 alunos**. Cada equipe terá a liberdade de escolher o tema, desde que atenda aos **requisitos mínimos** definidos. O **tema escolhido** pela equipe deve ser **apresentado** até o dia **22 de maio**. Após esse **prazo**, **não** será possível **trocar** o **tema**, e as **equipes** que **não apresentarem** um tema serão **atribuídas** um **tema obrigatório**.

A **apresentação** do trabalho será feita em **grupo** e a **defesa** do **projeto** será feita de forma **individual**, o **aluno** que **não** fizer a **defesa** **não** receberá **nota**.

Requisitos Mínimos do Projeto:

1. CRUD's (**Cadastro, Alteração, Deleção** e **Listagem**):

- O sistema deve ter no mínimo **3 CRUD's**, permitindo o cadastro, alteração, deleção e listagem de informações relevantes para o contexto do projeto.

2. Relação entre **CRUD's**:

- É necessário estabelecer uma relação entre pelo menos **2** dos **CRUD's**. Um deles deve possuir uma chave estrangeira para estabelecer essa relação.

3. Herança:

- O sistema deve apresentar uma superclasse que contenha atributos e métodos comuns a diferentes entidades do projeto, promovendo a reutilização de código e a organização da estrutura de classes.

4. Classe Abstrata:

- É preciso implementar pelo menos uma classe abstrata que sirva como base para outras classes. Essa classe abstrata deve conter pelo menos um método abstrato que será implementado nas classes filhas.

5. Polimorfismo de Sobrescrita:

- Deve-se utilizar o conceito de polimorfismo de sobrescrita, onde um método da classe pai é redefinido nas classes filhas para se adequar às suas particularidades.

6. Polimorfismo de Sobrecarga:

- Utilize o polimorfismo de sobrecarga ao implementar métodos com o mesmo nome, mas com assinaturas diferentes, para lidar com situações específicas do projeto.

7. Interface:

- O sistema deve conter pelo menos uma interface que defina um contrato a ser seguido pelas classes que a implementarem. Essa interface pode estabelecer métodos obrigatórios ou padrões a serem seguidos.

8. Encapsulamento:

- É fundamental aplicar os conceitos de encapsulamento, definindo os atributos e métodos como públicos, privados ou protegidos, de acordo com as necessidades e boas práticas de programação.

9. MVC (Model**, **View** e **Controller**):**

- Organize o projeto seguindo a arquitetura **MVC**, separando adequadamente a lógica de negócio (**Model**), a interface de usuário (**View**) e a camada de controle (**Controller**).

10. Utilizar estruturas de laço e controle:

- Utilize estruturas de laço (**for**, **foreach**, **while** ou **do while**) e estruturas de controle (**if**, **else if**, **else** ou **switch**) de maneira apropriada no sistema, proporcionando um fluxo lógico e eficiente.

11. Log:

- Crie um log que salve as informações do sistema, pode ser um log de erros ou de atividades que deve ser salvo em txt.

12. Serialização:

- Implemente o salvamento e recuperação dos objetos em arquivo.

13. Clean Code:

- Aplique os princípios de **Clean Code** para escrever um código claro, organizado e eficiente, facilitando sua manutenção e colaboração. Priorize nomes descritivos, estrutura modular, simplicidade e boas práticas de formatação para garantir qualidade e legibilidade